	Carátula para entrega de prácticas
Facultad de Ingeniería	Laboratorios de docencia

Laboratorio de Computación Salas A y B

Profesor(a): René Adrián Dávila Pérez.

Asignatura: Programación orientada a objetos.

Grupo: 07

No de Práctica(s): 9

Integrante(s): 323331229

426060019

323217156

312109239

323195386

321321976

No. de lista o brigada: 6

Semestre: 3ro

Fecha de entrega: 30/10/2026

Observaciones:

CALIFICACIÓN: _____

Índice

1. Introducción **3**
1.1. Planteamiento del problema 3
1.2. Motivación 3
1.3. Objetivos 3

2. Marco Teórico **4**

3. Desarrollo **6**
3.1. Práctica 5 - Ejercicio 2: 6
3.2. Práctica 5 - Ejercicio 3: 6
3.3. Práctica 7 - Ejercicio 2: 7
3.4. Práctica 8 - Ejercicio 2: Huella de Carbono 7
3.5. Proyecto (Etapa 2) - Sistema Hospitalario: 8

4. Resultados **9**

5. Conclusiones **16**

1. Introducción

1.1. Planteamiento del problema

Esta práctica se basa en la necesidad de documentar y formalizar la lógica de los sistemas orientados a objetos que construimos previamente. Permitiendo abstraer y visualizar las relaciones y jerarquías entre las clases y el seguimiento del comportamiento dinámico de los objetos en tiempo de ejecución mediante los diagramas de secuencia.

1.2. Motivación

Utilizar UML como herramienta para diseñar soluciones de software para un lenguaje de programación orientado a objetos. Esto nos permite evaluar la eficiencia de nuestros diseños y transmitir soluciones técnicas.

1.3. Objetivos

- **General:** Representar gráficamente los ejercicios de las Prácticas 5, 6, 7 y 8, así la Etapa 2.
- **Específicos:**
  - Elaborar los diagramas de clases de incluyendo la cardinalidad de las relaciones.
  - Crear los diagramas de secuencia que representen el flujo de todo el ejercicio completo.

2. Marco Teórico

1. **UML:** (Unified Modeling Language) Es un lenguaje de modelado estándar que se utiliza para visualizar un plan arquitectónico para elementos como actividades, procesos y esquemas de base de datos. Permite visualizar, especificar y documentar cada una de las partes que comprende el desarrollo de software. [2]

2. **Diagramas UML:** Están compuestos por elementos (representaciones de objetos) y por las relaciones entre ellos, permiten modelar aspectos conceptuales como procesos de negocios o funcionalidades de un sistema. [1]

3. **Diagramas estáticos o estructurales:** Aportan una visión fija del sistema. [1]

4. **Diagrama de caso de uso:** Define la manera en la que el usuario del sistema interactúa con éste. Los diagramas de casos de uso permiten modelar el comportamiento de un sistema, un subsistema o, incluso, una clase. Está compuesto por 3 elementos básicos: Actor(es), Caso(s) de uso, Relación(es) (uso, herencia y comunicación). [1]

5. **Diagrama de clases:** Un diagrama de clases permite modelar las características de las clases que componen al sistema. Dentro de cada clase se pueden visualizar los atributos y métodos que contiene la clase. El conjunto de clases permite observar las relaciones que existen entre ellas dentro del sistema. Los diagramas de clase están compuestos por 3 elementos básicos: Clase(s), Cardinalidad, Relación(es). [1]

6. **Diagrama de objetos:** Los diagramas de objetos modelan las instancias generadas a través de las clases y se utilizan para describir al sistema en un instante de tiempo (o acción) en particular. Permiten mostrar los objetos y las relaciones entre ellos en un momento dado, por lo tanto, representa la parte estática de la interacción entre objetos (una situación específica en un momento determinado). Los elementos utilizados para generar este tipo de diagramas son: Objeto(s), Asociación(es). [1]

7. **Diagrama de estados:** Los diagramas de estados describen las transiciones por las que puede pasar un objeto durante su tiempo de vida en la aplicación, así como la manera en la que cambia de estado el objeto. Los elementos de un diagrama de estados son: Nodos de entrada y salida, Estado(s) del objeto, Transición(es) entre estados. [1]

8. **Diagrama de actividades:** El diagrama de actividades muestra el flujo de acciones (operaciones que se ejecutan) y los objetos involucrados. Permite visualizar el orden en el que se van realizando tareas dentro de un sistema, así como los objetos involucrados en la comunicación. Los componentes básicos de un diagrama de actividades son: Nodos de entrada y salida, Actividad(es), Transiciones.[1]

9. **Diagrama de interacción:** Estos diagramas representan la comunicación que se lleva a cabo entre un cliente (actor) o un objeto (clase) cuando se ejecuta una acción en el sistema. Los elementos básicos de los diagramas de interacción son: Objeto o actor, Enlace(s), Mensaje(s). Los diagramas de interacción, a su vez, se dividen en dos tipos: Diagrama de secuencia, Diagrama de colaboración. [1]

10. **Diagrama de secuencia:** Los diagramas de secuencia muestran una interacción ordenada de eventos, visualizando los objetos participantes en cada interacción, así como los mensajes que intercambian entre ellos. Los diagramas de secuencia permiten ver la interacción que se genera al ejecutar una acción, así como los objetos involucrados en el proceso. [1]

11. **Diagrama de comunicación** Los diagramas de comunicación muestran la interacción entre varios objetos y el orden que existen entre ellos. La secuencia de los mensajes y los flujos de ejecución concurrentes se determinan mediante números secuenciales. [1]

3. Desarrollo

Para la realización de esta práctica, la implementación consistió en el análisis de las Prácticas 5, 7 y 8, así como en el avance del Proyecto (Etapa 2). El proceso de desarrollo se dividió en dos fases de abstracción para cada ejercicio:

- **Modelado Estructural (Diagramas de Clases):** Se analizaron los archivos para identificar las clases, interfaces y clases abstractas. Se representaron gráficamente los atributos y métodos con sus respectivos modificadores de acceso, así como las relaciones de herencia, implementación y asociación, definiendo cuidadosamente la cardinalidad entre los componentes del sistema.
- **Modelado Dinámico (Diagramas de Secuencia):** Se analizó el flujo de ejecución desde las clases `Main`. Se establecieron las líneas de vida de cada objeto instanciado y se trazó el intercambio de mensajes (llamadas a métodos) en orden cronológico, representando la interacción real de los componentes para cumplir los requerimientos del programa.

3.1. Práctica 5 - Ejercicio 2:

Diagrama de Clases: El modelo define dos entidades principales: MainEmpleado y Empleado. MainEmpleado actúa como la clase controladora y presenta una asociación dirigida hacia Empleado con una cardinalidad de 1 a 1, indicando que el flujo principal instancia y gestiona un único objeto de esta clase. La clase Empleado aplica el principio de encapsulamiento, manteniendo sus atributos nombre, apellido y salarioMensual como privados, cuenta con su constructor, métodos de acceso (getters), métodos modificadores (setters) y métodos (aumentarSalario y mostrar).

Diagrama de Secuencia: El modelo dinámico detalla la interacción iniciada por la línea de vida de MainEmpleado. El proceso arranca con un mensaje de instanciación (new Empleado) que pasa los parámetros iniciales para construir el objeto empleado1. Posteriormente, el flujo principal emite llamadas síncronas para extraer el estado del objeto mediante sus getters, recibiendo como respuesta los valores almacenados en memoria. Tras ejecutar el método mostrar(), la clase principal emite mensajes setters para actualizar el salario y el nombre. Finalmente, se invoca el método aumentarSalario(), verificando el resultado con una última llamada a mostrar().

3.2. Práctica 5 - Ejercicio 3:

Diagrama de Clases: Se compone de tres elementos: MainFecha, Scanner y Fecha. La clase MainFecha establece asociaciones dirigidas (con cardinalidad 1 a 1) con las otras dos clases, asumiendo el rol de orquestador, la clase Fecha estructura la información temporal protegiendo sus variables internas (dia, mes, año) y habilitando métodos getters y setters para su manipulación e impresión en consola. **Diagrama de Secuencia:** La secuencia de ejecución ilustra la comunicación entre la línea de vida central (MainFecha) y los objetos scanner y fecha. La interacción inicia instanciando el objeto Scanner para habilitar la lectura del flujo de entrada, MainFecha envía mensajes iterativos nextLine() al scanner para capturar

los datos ingresados por el usuario correspondientes al día, mes y año. Con estos datos en el flujo principal, se envía el mensaje de creación (new Fecha) para inicializar la instancia. El ciclo continúa con la extracción de datos iniciales mediante getters, la impresión en pantalla con mostrarFecha(), y una modificación forzada del estado interno utilizando los setters. Tras validar los cambios, el diagrama concluye enviando un mensaje close() al objeto scanner para asegurar el cierre del flujo y liberar los recursos del sistema.

3.3. Práctica 7 - Ejercicio 2:

Diagrama de Clases: Todo parte de la clase Punto, que encapsula las coordenadas (x,y) lo que provee el método base para calcular la distancia entre dos puntos mediante el teorema de Pitágoras. A partir de ahí, se define la superclase Cuadrilátero, la cual tiene una relación de agregación directa (cardinalidad) con 4 objetos de tipo Punto y establece los métodos generales mostrarPuntos(), calcularPerimetro() y una versión base de calcularArea(). El núcleo del diagrama de clases es la cadena de herencia que se despliega hacia abajo: la clase Trapezoide hereda de Cuadrilátero, Paralelogramo hereda de Trapezoide, Rectángulo hereda de Paralelogramo, y finalmente Cuadrado hereda de Rectángulo. Cada clase hija aplica polimorfismo sobrescribiendo (@Override) el método calcularArea() con la fórmula matemática correspondiente a su figura. Todo esto es orquestado por la clase Main, que presenta una dependencia hacia todas las figuras para poder instanciarlas.

Diagrama de Secuencia: La línea de vida principal, operada por Main, comienza enviando múltiples mensajes de creación (new) para instanciar los objetos geométricos (cuadrado, rectángulo, paralelogramo, trapezoide), pasándoles en el constructor cuatro instancias de Punto a cada uno. Posteriormente, se envía un mensaje de instanciación al Scanner para la entrada de datos, el diagrama entra en un ciclo do-while, cuya condición de salida es que la opción ingresada sea diferente de 5. Dentro del ciclo, Main se comunica con el scanner enviando el mensaje nextInt() para solicitar una opción al usuario. Esto desencadena en el switch, donde dependiendo del número elegido (1 a 4), el sistema dirige el flujo hacia una línea de vida específica (la figura geométrica seleccionada). Hacia dicha figura, se disparan tres mensajes secuenciales: mostrarPuntos(), calcularPerimetro() y calcularArea(), retornando los valores calculados al flujo principal para imprimirlos. Una vez que el usuario elige la opción 5, se rompe el ciclo y Main emite un último mensaje close() hacia la línea de vida del scanner para finalizar la ejecución.

3.4. Práctica 8 - Ejercicio 2: Huella de Carbono

Diagrama de Clases: El componente central es la interfaz «interface» HuellaCarbono, la cual establece un contrato obligando a que cualquier clase que la implemente defina el método getHuellaCarbono(). Creamos tres clases que implementaran esta interfaz: Auto, Edificio y Bicicleta. Cada una encapsula atributos totalmente distintos según su naturaleza (por ejemplo, el Auto maneja la distanciaRecorrida y el Edificio el consumo de electricidad y consumoGas), así como sus propias constantes de conversión ambiental (FACTOR_EMISION, FACTOR_ELECTRICO, etc.). La clase Main coordina el sistema agrupando estos objetos dispares mediante una estructura polimórfica List<HuellaCarbono> (ArrayList). **Diagrama de Secuencia:** La interacción arranca con una fase de inicialización donde el flujo princi-

pal envía mensajes sincrónicos (Llamada al constructor) de forma sucesiva para instanciar a auto1, edificio1 y bicicleta1, recibiendo en cada paso el retorno de la creación. Una vez que los objetos existen y son agrupados en la lista iterable en el código base, Main comienza a interactuar con cada una de las líneas de vida. El patrón de comunicación es idéntico para los tres objetos: primero lanza un mensaje toString() para extraer la cadena descriptiva de la instancia, e inmediatamente después envía el mensaje getHuellaCarbono() para ejecutar el cálculo matemático interno del objeto.

3.5. Proyecto (Etapa 2) - Sistema Hospitalario:

Diagrama de Clases: La clase base Sistema actúa como el gestor central del hospital, almacenando en sus atributos privados colecciones tipo List<> con cardinalidad de 1 a muchos (1..* y 0..*) hacia tres clases abstractas fundamentales: Medico, Enfermero y Paciente. Cada una de estas clases abstractas implementa su respectiva interfaz («interface» AtencionMedica, AtencionEnfermero, AtencionPaciente) y define atributos protegidos () para ser heredados. A su vez, el sistema aplica herencia para especializar el comportamiento: Medico se divide en MedicoEspecialista y MedicoGeneral; Enfermero en EnfermeroEspecialista y JefeEnfermeria; y Paciente en PacienteEspecialidad y PacienteGeneral. Finalmente, la clase Main mantiene una relación de uso directa hacia Sistema para arrancar y orquestar toda la lógica del hospital.

Diagrama de Secuencia: Inicia en la línea de vida Main interactuando con scanner para capturar el nombre del hospital y lanzar el mensaje de creación new Sistema(nombreHospital). Seguido de esto, el flujo principal instancia secuencialmente: un MedicoEspecialista, un MedicoGeneral, un JefeEnfermeria, un EnfermeroEspecialista y dos pacientes. A partir de ahí, la interacción se divide en tres bloques principales gestionados a través del objeto hospital:Métodos de Registro: Main envía mensajes como registroEnSistema(hospital) a los objetos creados, los cuales internamente lanzan mensajes de vuelta al hospital usando métodos como registroMedico(this), registroEnfermero(this) y registroPaciente(this) para guardarse en las listas del sistema. Asignación de Pacientes: Main ordena al hospital vincular mediante el mensaje asignarPaciente(). El hospital delega la acción enviando el mensaje agregarPaciente(paciente1) a la línea de vida del médico o enfermero correspondiente. Flujo de Consulta y Tratamiento: Se simula la operación médica. Main envía darConsulta(paciente1) al hospital, provocando que el estado interno del paciente cambie (mensaje setEnConsulta(true) hacia el paciente). Luego se finaliza la consulta y se asigna un tratamiento (setEnTratamiento(true)), repitiendo el proceso para el paciente 2. El flujo termina con el cierre del scanner (close()).

4. Resultados

A continuación se presentan los diagramas de clases y de secuencia obtenidos como resultado del proceso de abstracción y modelado de las Prácticas 5, 7, 8 y el avance del Proyecto (Etapa 2).

Práctica 5 - Ejercicio 2: Empleado

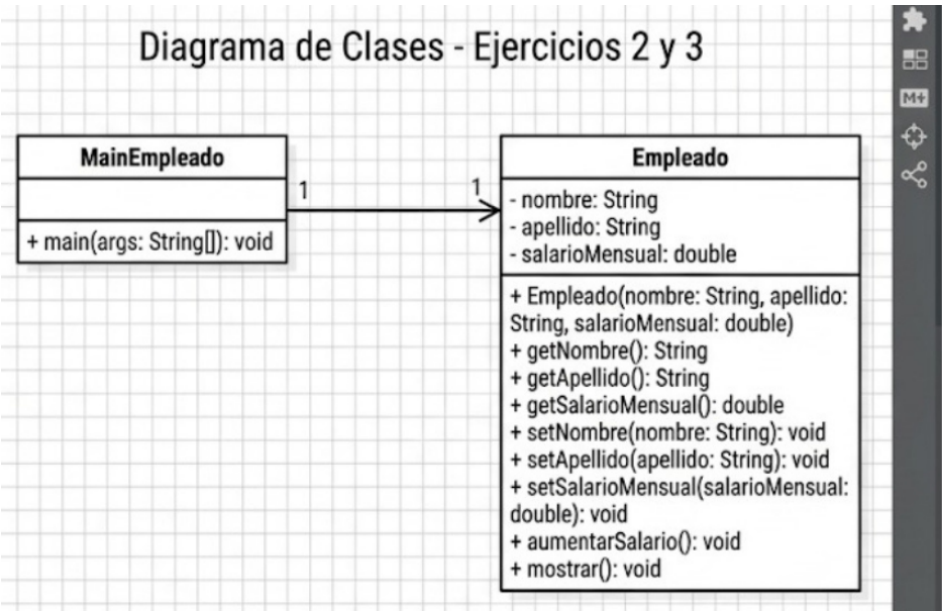


Figura 1: **Práctica 5.** Diagrama de clases con cardinalidad representando la estructura de Empleado y MainEmpleado.

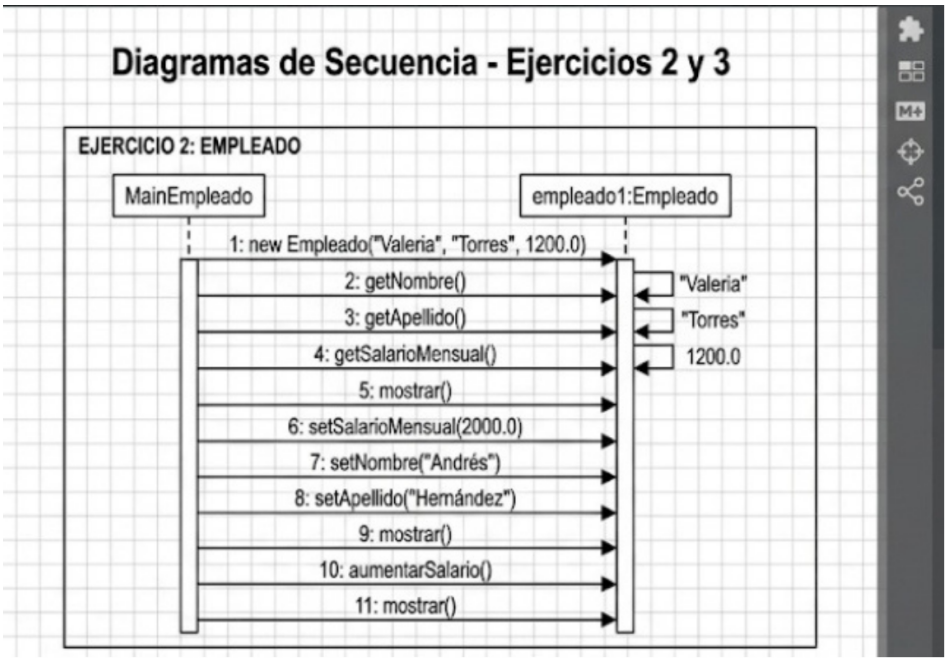


Figura 2: **Práctica 5.** Diagrama de secuencia representando el flujo de instanciación y ejecución de métodos del objeto Empleado.

Práctica 5 - Ejercicio 3: Fecha

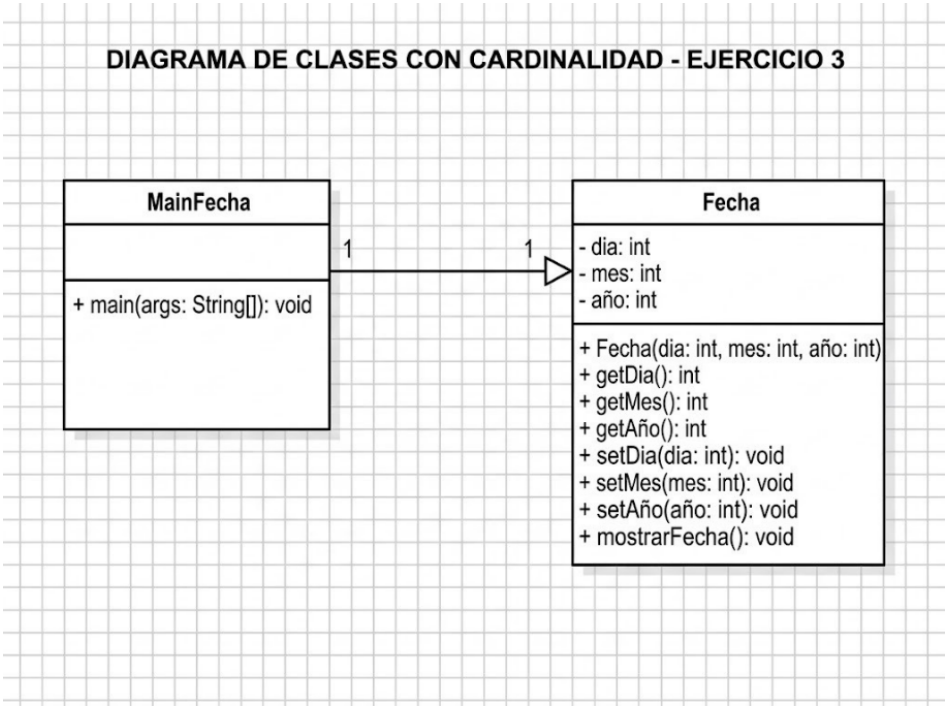


Figura 3: **Práctica 5.** Diagrama de clases con cardinalidad representando la estructura de Fecha y la interacción con Scanner.

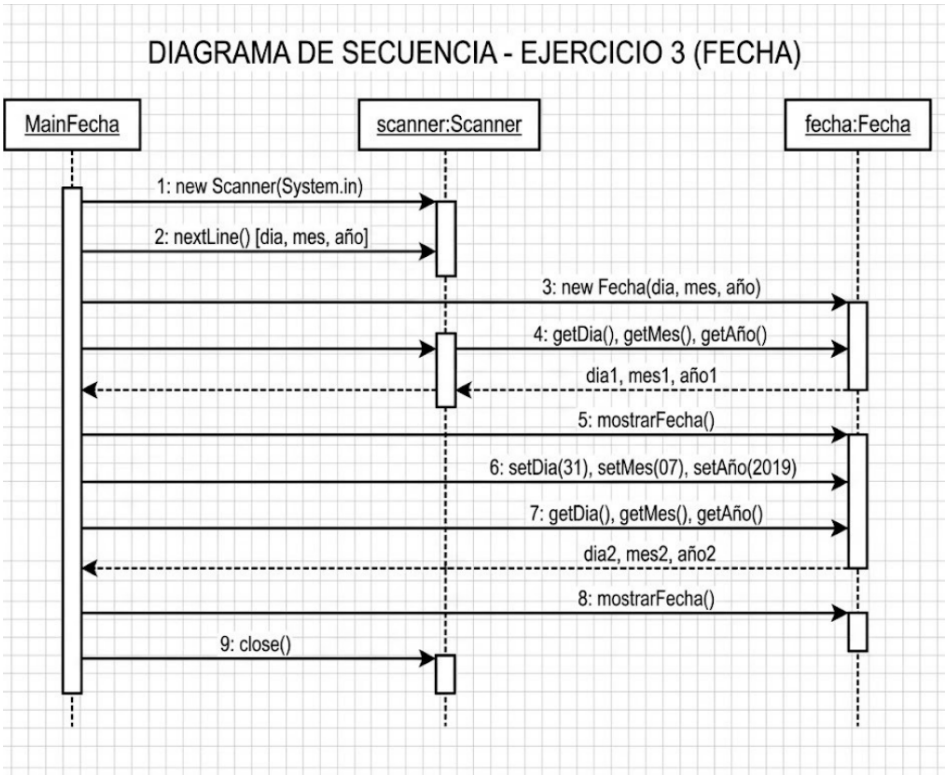


Figura 4: **Práctica 5.** Diagrama de secuencia representando la entrada de datos y creación del objeto Fecha.

Práctica 7 - Ejercicio 2: Figuras Geométricas

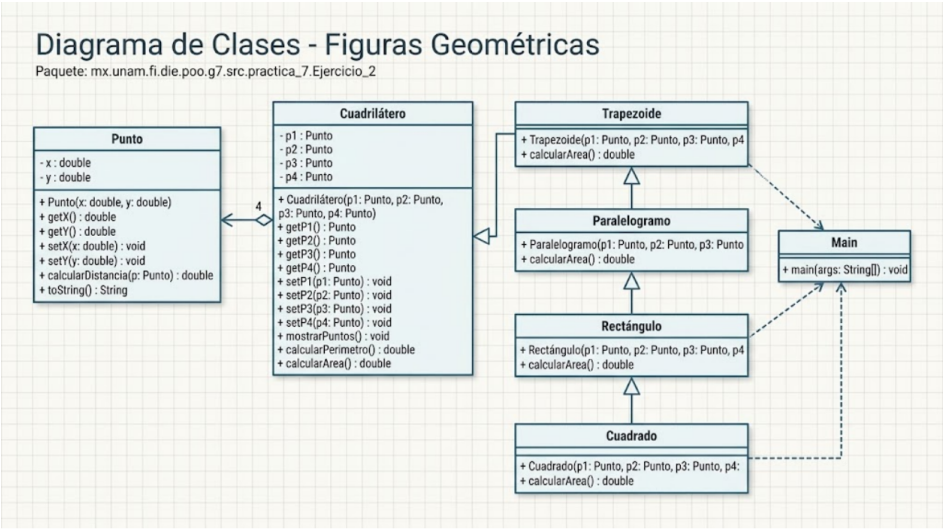


Figura 5: **Práctica 7.** Diagrama de clases con cardinalidad representando la jerarquía y estructura de las figuras geométricas.

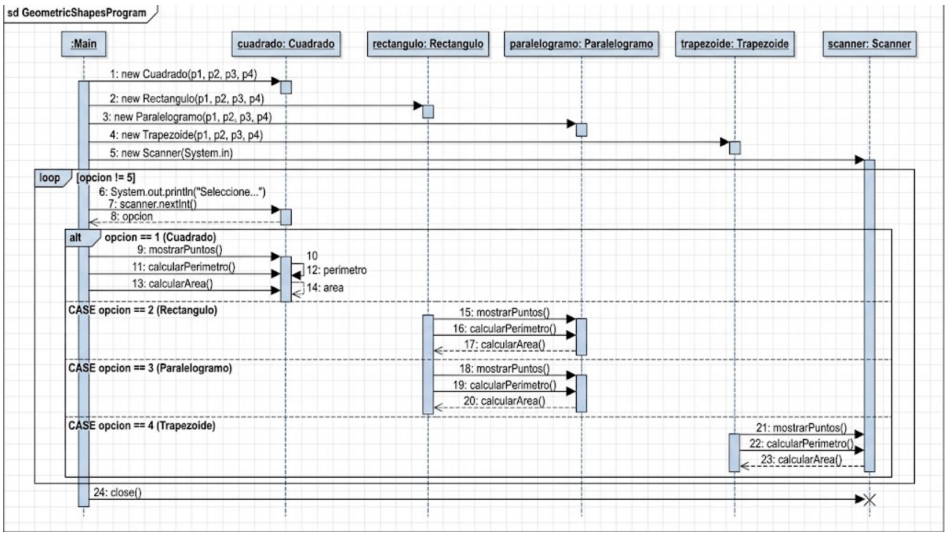


Figura 6: **Práctica 7.** Diagrama de secuencia que modela el menú interactivo y la instanciación polimórfica de las figuras.

Práctica 8 - Ejercicio 2: Huella de Carbono

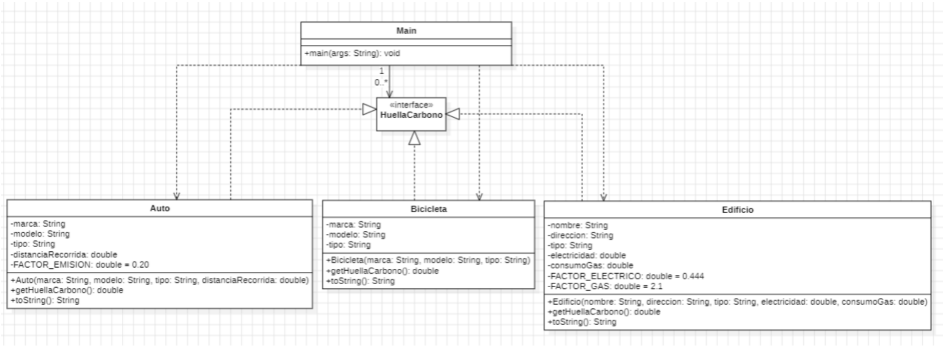


Figura 7: **Práctica 8.** Diagrama de clases con cardinalidad representando la implementación de la interfaz HuellaCarbono.

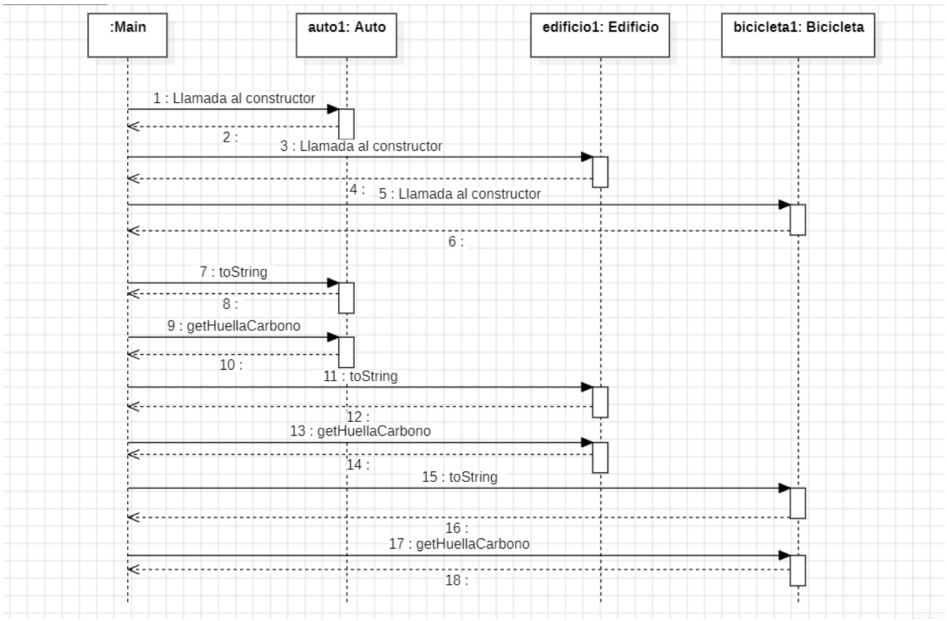


Figura 8: **Práctica 8.** Diagrama de secuencia mostrando la ejecución polimórfica del cálculo de emisiones para distintos objetos.

Proyecto (Etapa 2) - Sistema Hospitalario POO

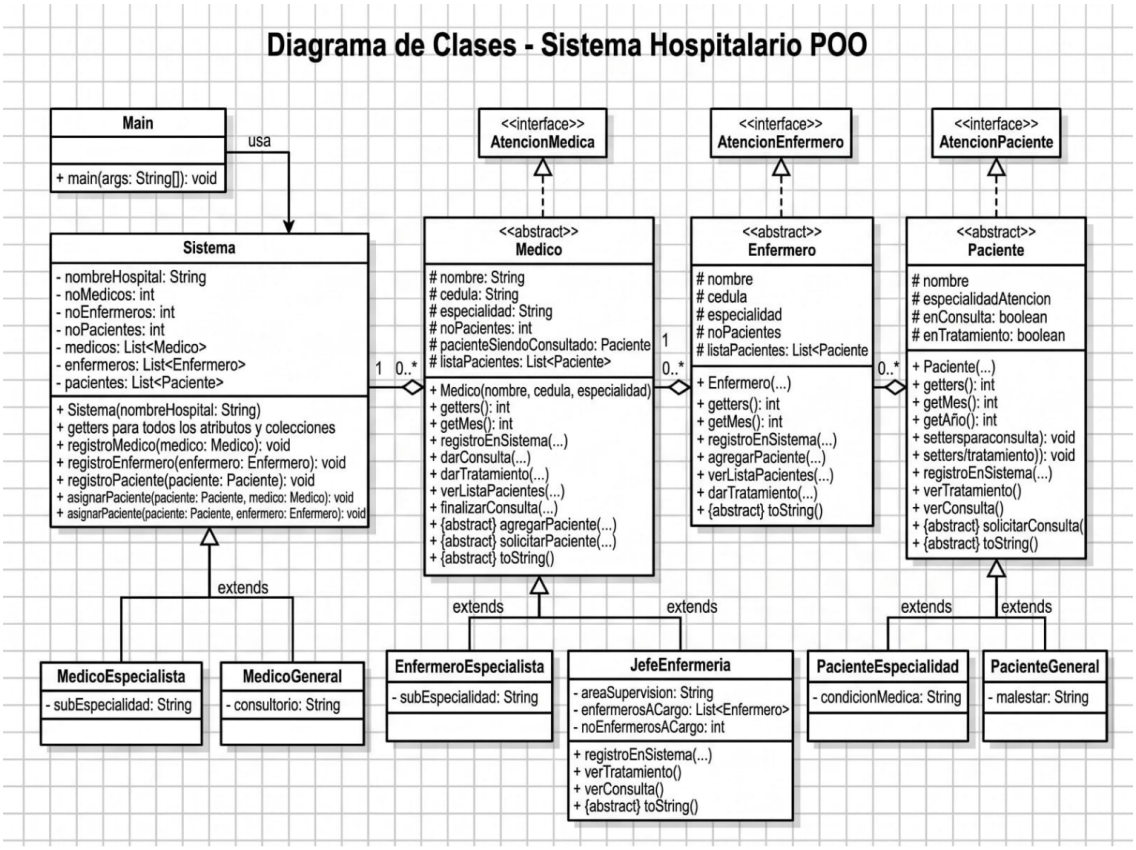


Figura 9: **Etapa 2.** Diagrama de clases con cardinalidad representando la jerarquía, interfaces y estructura del Sistema Hospitalario.

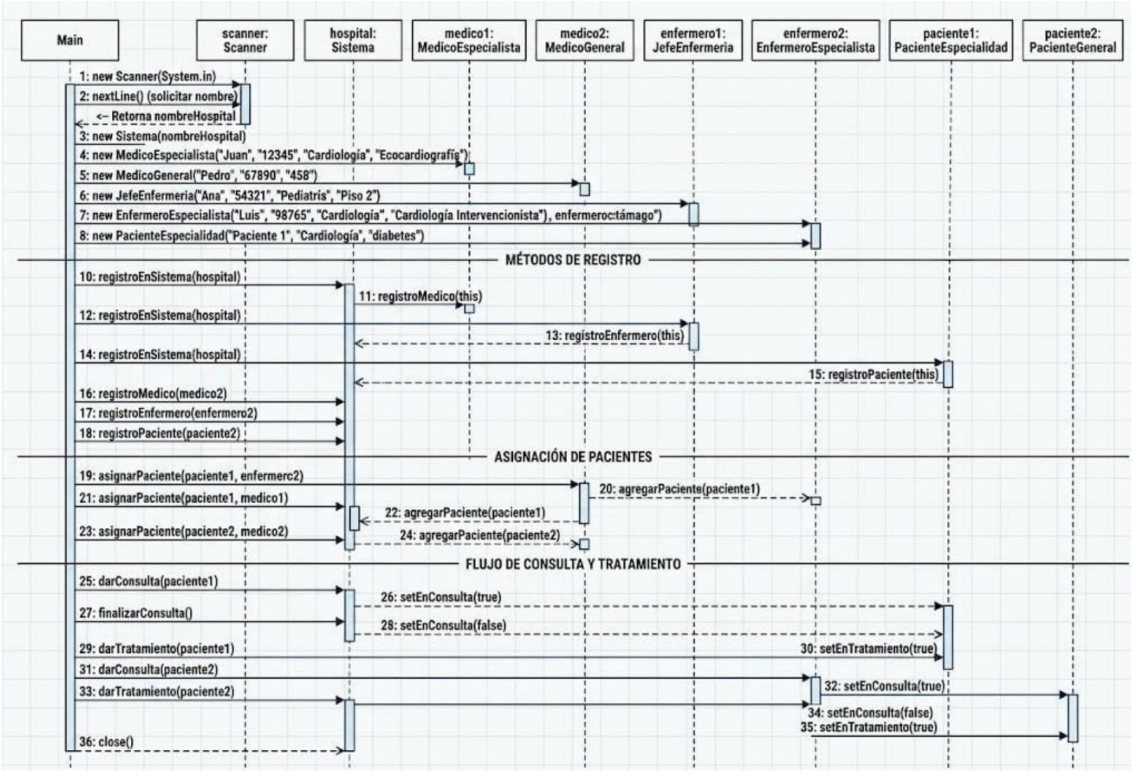


Figura 10: **Etapa 2.** Diagrama de secuencia representando el flujo completo de registro, asignación y consultas en el hospital.

5. Conclusiones

El análisis de los resultados obtenidos demuestra la importancia de aplicar los conceptos de modelado UML para resolver el problema de documentar. Al trasladar el código fuente a diagramas de clases y de secuencia, se pudo comprobar visualmente que la lógica diseñada en las prácticas anteriores tiene una estructura sólida y coherente. Esto resalta que estas herramientas de modelado no son un simple esquema gráfico, sino un pilar fundamental que permite entender la interacción real, verificar la eficiencia y garantizar que la solución planteada cumple su propósito.

Referencias

[1] Gálvez, J. A. S. (s.f.-b). UML. Unidades de Apoyo Para el Aprendizaje - CUAED - UNAM. https://repositorio-uapa.cuaed.unam.mx/repositorio/moodle/pluginfile.php/3072/mod_resource/content/1/UAPA-UML/index.html

[2] Modelo UML (Unified Modeling Language). (2021, 28 febrero). IBM. Recuperado 29 de octubre de 2026, de https://www.ibm.com/docs/es/iis/11.5.0?topic=types-unified-modeling-language-uml-model